1. Aufgabe: Versionierung/Versionsverwaltung (10 Punkte)

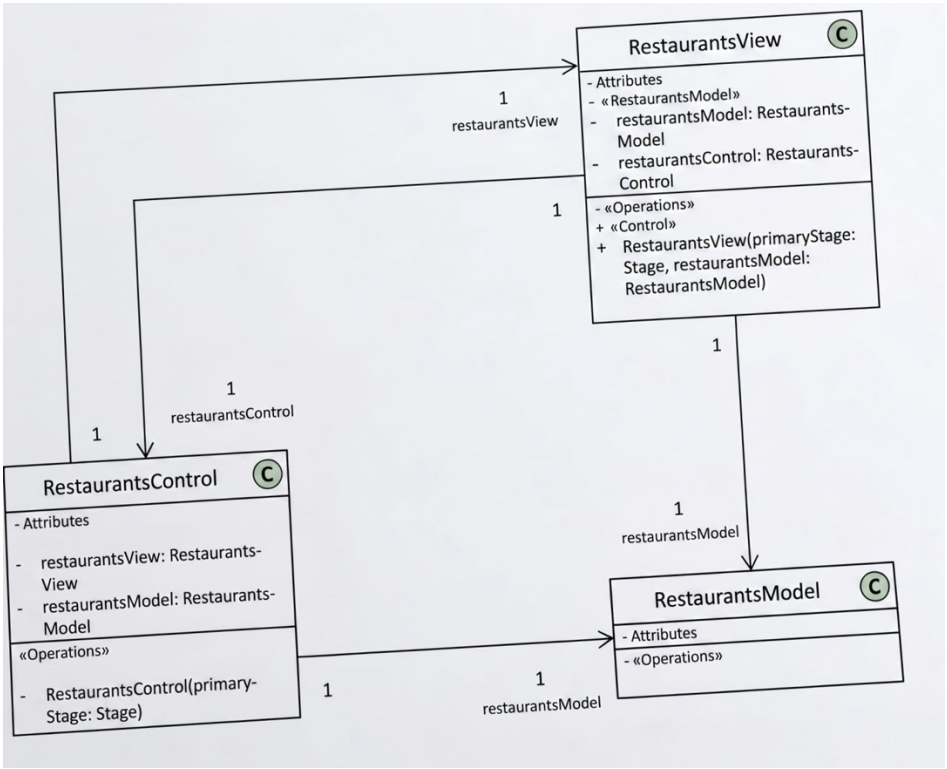
Erläutern Sie kurz (in jeweils 2-3 Sätzen):

- Was versteht man im Zusammenhang mit Versionsverwaltung unter Merge?

- Was versteht man im Zusammenhang mit Versionsverwaltung unter Einchecken mit einem Konflikt?

2. Aufgabe: Design Patterns (45 Punkte)

Gegeben ist ein UML-Klassendiagramm zum Entwurfsmuster **Model View Control (MVC)**. Dieses soll für die Struktur einer Anwendung zur Verwaltung von Restaurants angewendet werden.



Gegeben sind weiterhin die folgenden Main, Restaurant und RestaurantsView.

Klasse Main

```
import javafx.application.Application;
import javafx.stage.Stage;

public class Main extends Application {

    @Override
    public void start(Stage primaryStage) {
        new RestaurantsControl(primaryStage);
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Klasse Restaurant

```
public class Restaurant {

    private String name;
    private int anzahlTische;
    private int eroeffnungsjahr;
    private int jahrLetzteRenovierung;

    // Standardkonstruktor zur Belegung der Attribute und
    // get- und set-Methoden

    public String gibAttributeZurueck() {
        return this.getName() + ", " + this.getAnzahlTische()
                + " Tische, Eroeffnungsjahr: "
                + this.getEroeffnungsjahr()
                + ", letzte Renovierung: "
                + this.getJahrLetzteRenovierung();
    }
}
```

Klasse RestaurantsView

```
//Importe

public class RestaurantsView {
    // Hier ergaenzen!




    private GridPane grid            = new GridPane();
    private TextField txtRestaurants = new TextField();
    private Button btnRestaurants    = new Button("Restaurants lesen");

    public RestaurantsView(
        /* Hier ergaenzen! */


        )
    {

        // Hier ergaenzen!
```

```
            this.grid.setAlignment(Pos.CENTER);
            this.grid.setHgap(15);
            Scene scene = new Scene(grid, 500, 150);
            primaryStage.setScene(scene);
            primaryStage.show();
            this.initKomponenten();
            this.initListener();
        }

    private void initKomponenten() {
        this.txtRestaurants.setPrefWidth(330);
        this.grid.add(this.txtRestaurants, 0, 0);
        this.grid.add(this.btnRestaurants, 1, 0);
    }

    private void initListener() {
        btnRestaurants.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent e) {

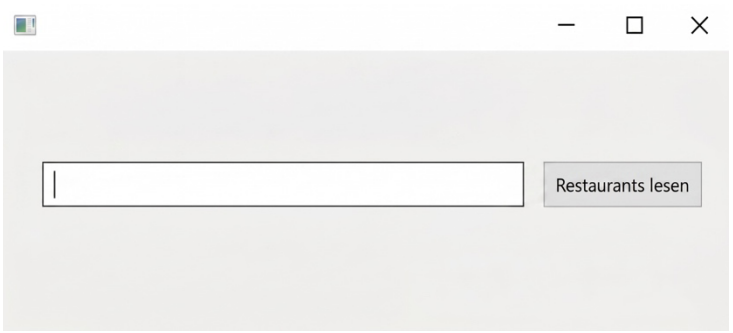
            }
        });
    }

    void setTxtRestaurants(String text) {

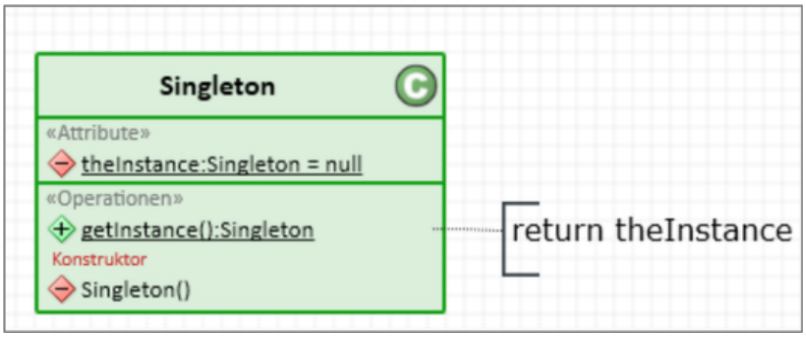
    }

    void zeigeFehlermeldungAn(String meldung) {
        Alert alert = new Alert(AlertType.ERROR);
        alert.setTitle("Fehlermeldung");
        alert.setContentText(meldung);
        alert.showAndWait();
    }
}
```

a) Geben Sie den Quellcode an, der aus dem UML-Klassendiagramm zu dem Pattern *Model View Control* hervorgeht, beziehungsweise ergänzen Sie dafür die bereits vorliegende Klasse `RestaurantsView` an den gekennzeichneten Stellen, so wie Sie es in der Veranstaltung kennengelernt haben. Packages und Importe brauchen Sie nicht zu berücksichtigen. Beim Start der Anwendung soll das folgende Fenster aufgeblendet werden.



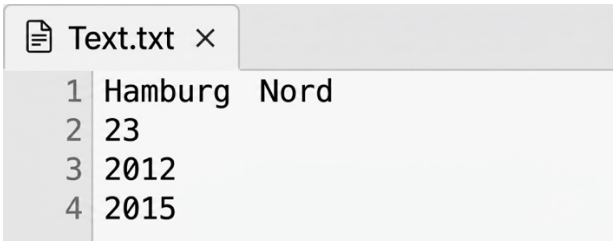
b) Singleton-Pattern
Gegeben ist ein UML-Klassendiagramm zum Entwurfsmuster **Singleton**.



Geben Sie den Quellcode an, der implementiert werden muss, damit garantiert ist, dass maximal ein Objekt vom Typ `ResturantModel` kreiert wird. Nennen Sie auch die Klassen und Methoden zu welcher der jeweilige Quellcode gehört.

Platz für den zu ergänzenden Quellcode

c) Geben Sie weiterhin den Quellcode an, der ergänzt werden muss, um unter der Berücksichtigung von *Model View Control*, wie Sie es in der Veranstaltung kennengelernt haben, die folgende Funktionalität zu erhalten. Geben Sie an, in welcher Klasse und gegebenenfalls in welcher Methode der Quellcode ergänzt werden muss!
Beim Klick auf den Button mit der Aufschrift *Restaurants lesen* werden die Daten zu einem Restaurant aus der Textdatei *Text.txt* gelesen und in dem Textfeld angezeigt. Die Auswertung von Fehlerfällen soll im Control durchgeführt werden. Es soll im Fehlerfall eine Fehlermeldung erscheinen.



Hinweis: Quellcodeausschnitt zum Lesen von Daten aus der Datei:

```
BufferedReader ein =
        new BufferedReader(new FileReader("Text.txt"));
this.restaurant = new Restaurant(
        ein.readLine(),
        Integer.parseInt(ein.readLine()),
        Integer.parseInt(ein.readLine()),
        Integer.parseInt(ein.readLine())
);
ein.close();
```

Platz für den zu ergänzenden Quellcode

3. Aufgabe: Vertiefung Java (35 Punkte)

Gegeben ist die Klasse Restaurant wie in der obigen Aufgabe. Gegeben ist weiterhin eine Klasse RestaurantRestaurantDrivein und eine Klasse Anwendersystem, Packages und Importe werden hier nicht berücksichtigt.

```
public class RestaurantRestaurantDrivein extends Restaurant {

    private boolean mitRestaurantDrivein;

    public RestaurantRestaurantDrivein(String name, int anzahlTische,
                                       int eroeffnungsjahr,
                                       int jahrLetzteRenovierung,
                                       boolean mitRestaurantDrivein)
    {
        super(name, anzahlTische, eroeffnungsjahr, jahrLetzteRenovierung);
        this.mitRestaurantDrivein = mitRestaurantDrivein;
    }

    public boolean isMitRestaurantDrivein() {
        return mitRestaurantDrivein;
    }

    public void setMitRestaurantDrivein(boolean mitRestaurantDrivein) {
        this.mitRestaurantDrivein = mitRestaurantDrivein;
    }

    @Override
    public String gibAttributeZurueck() {
        return super.gibAttributeZurueck()
                + ", mit Drivein: " + this.isMitRestaurantDrivein();
    }
}

public class Anwendersystem {

    public static void main(String[] args) {
        Restaurant RestaurantHa =
                       new Restaurant("Hamburg Nord", 23, 2012, 2015);

        RestaurantRestaurantDrivein RestaurantBr
                = new RestaurantRestaurantDrivein(
                "Bremen West", 27, 1908, 2016, true);
    }
}
```

a) **Generics, for each**
Geben Sie folgend den zu ergänzenden Quellcode der main-Methode der Klasse Anwendersystem an, um ein Objekt vom Typ `ArrayList` unter Verwendung von Generics zu erzeugen.
Die Liste soll die beiden Objekte RestaurantHa und RestaurantBr, siehe Quellcode oben, erhalten. Geben Sie den Quellcode unter Verwendung einer for each-Schleife an, so dass Sie die folgende Ausgabe erhalten. (Mit `for each` ist nicht die Verwendung einer Methode forEach gemeint!)

```
Hamburg Nord, 23 Tische, Erstellungsjahr: 2012, letzte Renovierung 2015
Bremen West, 27 Tische, Erstellungsjahr: 2008, letzte Renovierung 2016, mit Drivein: true
```

Platz für den zu ergänzenden Quellcode

b) Generics mit Typeinschränkungen
Geben Sie den Quellcode einer Klasse `RestaurantUtil` an, welche ein Generic `T` enthält, für welches nur ein Objekt vom Typ `Restaurant` oder einer Ableitung von `Restaurant` eingesetzt werden kann. Es gibt ein privates Attribut `restaurantBasis` vom Typ `T`, einen Konstruktor, welcher das Attribut mit einem Wert belegt, und eine Methode `gibRestaurantBasisAus` ohne Parameter und Rückgabewert. Diese schreibt die Attribute von `T` in die Konsole, siehe unten.

Ergänzen Sie die `main`-Methode der Klasse `Anwendersystem` folgendermaßen. Erstellen Sie zwei Objekte vom Typ `RestaurantUtil` unter Verwendung der Objekte `RestaurantHa` und `RestaurantBr` und erzeugen Sie die folgende Ausgabe:

```
Attribute: Hamburg Nord, 23 Tische, Erstellungsjahr: 2012, letzte Renovierung 2015
Attribute: Bremen West, 27 Tische, Erstellungsjahr: 2008, letzte Renovierung 2016, mit Drivein: true
```

c) Funktionale Programmierung

Ergänzen Sie die `main`-Methode der Klasse `Anwendersystem` folgendermaßen. Geben Sie den Quellcode an, um ein Objekt vom Typ `Supplier<String>` und eines vom Typ `BooleanSupplier` zu erstellen. Benutzen Sie dazu jeweils das Objekt `RestaurantBr`, geeignete Methoden aus den Klassen `Restaurant` und `RestaurantRestaurantDrivein` und Lambda-Ausdrücke.

Geben Sie dann den Quellcode an, um mit Hilfe der beiden Supplier-Objekte die folgende Ausgabe in der Konsole zu erhalten:

```
Hat das Restaurant Bremen West einen Drivein? true
```

4. Aufgabe: JUnit und Tests (30 Punkte)

Gegeben sind die Klassen aus den vorigen Aufgaben.

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests unter Nutzung von `Supplier<String>` und aus der Klasse `Assertions` die Methoden wie `assertTrue(BooleanSupplier conditionSupplier, Supplier<String> messageSupplier)`, `assertEquals(String actual, String expected)` und `<T> assertThrows(Class <T> clazz, Executable executable)`

- Die Methode `setJahrLetzteRenovierung` der Klasse `Restaurant` soll durch diese JUnit Testklasse getestet werden. Es soll insbesondere getestet werden, dass der übergebene Wert nicht kleiner ist als das Erstellungsjahr.
- Ebenso soll der Konstruktor der Klasse `Restaurant` soll daraufhin getestet werden, ob Jahreseingaben richtig gesetzt werden, insbesondere soll auch hier die obige Bedingung wie für `setJahrLetzteRenovierung` getestet werden.
- Weiterhin sollen beide obigen Methoden daraufhin getestet werden, ob bei fehlerhafter Eingabe entgegen der oben getesteten Bedingung eine `IllegalArgumentException` mit der Meldung *"Die letzte Renovierung darf nicht später als das Eröffnungsjahr sein" geworfen wird*.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;
public class RestaurantTest {
    // Ergaenzen!!!



    @BeforeEach
    public void setUp() throws Exception {
    // Ergaenzen!!!

    }

    @AfterEach
    public void tearDown() throws Exception {
    // Ergaenzen!!!

    }

    @Test
    public void test() throws Exception {
    // Ergaenzen!!!




    }
}
```